Introduction to Artificial Intelligence

Search III: Adversarial Search and Games

Matteo Acclavio

05/03/2026

Plan

So far:

- We have seen how to describe an agent and its environment
- We have seen how a single agent can plan

Today:

- how to plan with multiple agents
- how to optimize search
- how to deal with uncertainty in the environment

How to model multiple agents?

Three main approaches:

- Economy:

How to model multiple agents?

Three main approaches:

- Economy: look at the behavior of all agents as a whole;
- Environment:

How to model multiple agents?

Three main approaches:

- Economy: look at the behavior of all agents as a whole;
- Environment: all agents except one are part of the environment;
- Explicit:

How to model multiple agents?

Three main approaches:

- Economy: look at the behavior of all agents as a whole;
- Environment: all agents except one are part of the environment;
- Explicit: model *adversaral games*

Simple Games

We mostly focus on 'simple' games, which are:

- Two-player: only two agents are playing the game

Simple Games

We mostly focus on 'simple' games, which are:

- Two-player: only two agents are playing the game
- Turn-based: players take turns to make their moves (discrete time)

Simple Games

We mostly focus on 'simple' games, which are:

- Two-player: only two agents are playing the game
- Turn-based: players take turns to make their moves (discrete time)
- Zero-sum: no win-win situation, one player's gain is the other player's loss

Simple Games

We mostly focus on 'simple' games, which are:

- Two-player: only two agents are playing the game
- Turn-based: players take turns to make their moves (discrete time)
- Zero-sum: no win-win situation, one player's gain is the other player's loss
- Perfect information: fully observable

Simple Games

We mostly focus on 'simple' games, which are:

- Two-player: only two agents are playing the game
- Turn-based: players take turns to make their moves (discrete time)
- Zero-sum: no win-win situation, one player's gain is the other player's loss
- Perfect information: fully observable

Examples:

- Tic-tac-toe
- Connect Four
- Checkers
- Chess
- Go

. . . well, not so ‘simple’

. . . well, not so ‘simple’

Number of states

- Tic-tac-toe:

. . . well, not so ‘simple’

Number of states

- Tic-tac-toe: $3^9 = 19683$ possible states
- Connect Four:

. . . well, not so ‘simple’

Number of states

- Tic-tac-toe: $3^9 = 19683$ possible states
- Connect Four: 4.531.985.219.092 possible states
- Checkers:

. . . well, not so ‘simple’

Number of states

- Tic-tac-toe: $3^9 = 19683$ possible states
- Connect Four: 4.531.985.219.092 possible states
- Checkers:around 10^{20} possible states
- Chess:

. . . well, not so ‘simple’

Number of states

- Tic-tac-toe: $3^9 = 19683$ possible states
- Connect Four: 4.531.985.219.092 possible states
- Checkers:around 10^{20} possible states
- Chess:between 10^{43} and 10^{47} possible states (around 10^{120} possible games)
- Go:

. . . well, not so ‘simple’

Number of states

- Tic-tac-toe: $3^9 = 19683$ possible states
- Connect Four: 4.531.985.219.092 possible states
- Checkers:around 10^{20} possible states
- Chess:between 10^{43} and 10^{47} possible states (around 10^{120} possible games)
- Go: around 10^{170} possible states

. . . well, not so 'simple'

Number of states

- Tic-tac-toe: $3^9 = 19683$ possible states
- Connect Four: 4.531.985.219.092 possible states
- Checkers:around 10^{20} possible states
- Chess:between 10^{43} and 10^{47} possible states (around 10^{120} possible games)
- Go: around 10^{170} possible states

Reminder: number of atoms in the observable universe is estimated between 10^{78} and 10^{82}

Two-player Games, mathematically

Game $= \langle S, A, T, U, P \rangle$ where

- S is the set of states
- S_0 is the initial state
- S_g is the set of goal states
- A is the set of actions
- $T : S \times A \rightarrow S$ is the transition function
- $U : S \rightarrow \mathbb{R}$ is the utility function
- $P : S \rightarrow \{\circ, \bullet\}$ is the player function [1]

[1] In the book, $\circ = MAX$ and $\bullet = MIN$.

Two-player Games, mathematically

Game $= \langle S, A, T, U, P \rangle$ where

- S is the set of states
- S_0 is the initial state
- S_g is the set of goal states
- A is the set of actions
- $T : S \times A \rightarrow S$ is the transition function
- $U : S \rightarrow \mathbb{R}$ is the utility function
- $P : S \rightarrow \{\circ, \bullet\}$ is the player function [1]

Winning Strategy (for a player) = tree of moves that guarantees a win

[1] In the book, $\circ = MAX$ and $\bullet = MIN$.

Two-player Games, mathematically

Game = $\langle S, A, T, U, P \rangle$ where

- S is the set of states
- S_0 is the initial state
- S_g is the set of goal states
- A is the set of actions
- $T : S \times A \rightarrow S$ is the transition function
- $U : S \rightarrow \mathbb{R}$ is the utility function
- $P : S \rightarrow \{\circ, \bullet\}$ is the player function [1]

Winning Strategy (for a player) = tree of moves that guarantees a win

Note: search in a two-player game = search in a single-player game with non-deterministic transitions (the opponent's moves are part of the environment)

[1] In the book, $\circ$ = *MAX* and $\bullet$ = *MIN*.

How to make decisions in a two-player game?

Naïve idea: check all possible moves, and pick the one that leads to the best outcome for the player

How to make decisions in a two-player game?

Naïve idea: check all possible moves, and pick the one that leads to the best outcome for the player

A naïve implementation:

- take the tree of the game,
- label the leaves with the utility of the corresponding state, and
- label the internal nodes with the maximum (if ∘'s turn) or the minimum (if •'s turn) of the values of their children.
- pick the move that leads to the child with the highest value.

How to make decisions in a two-player game?

Naïve idea: check all possible moves, and pick the one that leads to the best outcome for the player

A naïve implementation:

- take the tree of the game,
- label the leaves with the utility of the corresponding state, and
- label the internal nodes with the maximum (if ∘'s turn) or the minimum (if •'s turn) of the values of their children.
- pick the move that leads to the child with the highest value.

Why is this suboptimal?

Minimax Pseudocode

```
1: function Minimax(node)
2:     if node is a leaf then
3:         return U(node)
4:     else if node is a max node then
5:         v ← −∞
6:         for each child c of node do
7:             v ← max(v, Minimax(c))
8:         end for
9:         return v
10:    else
11:        v ← +∞
12:        for each child c of node do
13:            v ← min(v, Minimax(c))
14:        end for
15:        return v
16:    end if
17: end function
```

Minimax Search

- Optimal

Minimax Search

- Optimal if the opponent also plays optimally

Minimax Search

- Optimal if the opponent also plays optimally
- Complexity:

Minimax Search

- Optimal if the opponent also plays optimally
- Complexity: $O(b^m)$ where b is the branching factor and m is the maximum depth of the tree

Minimax Search

- Optimal if the opponent also plays optimally
- Complexity: $O(b^m)$ where b is the branching factor and m is the maximum depth of the tree
- Space complexity:

Minimax Search

- Optimal if the opponent also plays optimally
- Complexity: $O(b^m)$ where b is the branching factor and m is the maximum depth of the tree
- Space complexity: $O(bm)$

Minimax Search

- Optimal if the opponent also plays optimally
- Complexity: $O(b^m)$ where b is the branching factor and m is the maximum depth of the tree
- Space complexity: $O(bm)$

Can we do better?

Alpha-Beta (minimax) Search

```
1: function AlphaBeta(node, α, β)
2:     if node is a leaf then
3:         return U(node)
4:     end if
5:     if node is a max node then
6:         v ← −∞
7:         for each child c of node do
8:             v ← max(v, AlphaBeta(c, α, β))
9:             α ← max(α, v)
10:            if α ≥ β then
11:                return v
12:            end if
13:        end for
14:    else
15:        v ← +∞
16:        for each child c of node do
17:            v ← min(v, AlphaBeta(c, α, β))
18:            β ← min(β, v)
19:            if α ≥ β then
20:                return v
21:            end if
22:        end for
23:    end if
24:    return v
25: end function
```

Alpha-Beta and Minimax comparison

```
1: function Minimax(node)
2:     if node is a leaf then
3:         return U(node)
4:     else if node is a max node then
5:         v ← −∞
6:         for each child c of node do
7:             v ← max(v, Minimax(c))
8:         end for
9:         return v
10:    else
11:        v ← +∞
12:        for each child c of node do
13:            v ← min(v, Minimax(c))
14:        end for
15:        return v
16:    end if
17: end function
```

```
1: function AlphaBeta(node, α, β)
2:     if node is a leaf then
3:         return U(node)
4:     end if
5:     if node is a max node then
6:         v ← −∞
7:         for each child c of node do
8:             v ← max(v, AlphaBeta(c, α, β))
9:             α ← max(α, v)
10:            if α ≥ β then
11:                return v
12:            end if
13:        end for
14:    else
15:        v ← +∞
16:        for each child c of node do
17:            v ← min(v, AlphaBeta(c, α, β))
18:            β ← min(β, v)
19:            if α ≥ β then
20:                return v
21:            end if
22:        end for
23:    end if
24:    return v
25: end function
```

Alpha-Beta (minimax) Search

- Optimal

Alpha-Beta (minimax) Search

- Optimal if the opponent also plays optimally

Alpha-Beta (minimax) Search

- Optimal if the opponent also plays optimally
- Complexity:

Alpha-Beta (minimax) Search

- Optimal if the opponent also plays optimally
- Complexity: $O(b^{m/2})$ in the best case, $O(b^m)$ in the worst case

Alpha-Beta (minimax) Search

- Optimal if the opponent also plays optimally
- Complexity: $O(b^{m/2})$ in the best case, $O(b^m)$ in the worst case
- Space complexity:

Alpha-Beta (minimax) Search

- Optimal if the opponent also plays optimally
- Complexity: $O(b^{m/2})$ in the best case, $O(b^m)$ in the worst case
- Space complexity: $O(bm)$

Alpha-Beta (minimax) Search

- Optimal if the opponent also plays optimally
- Complexity: $O(b^{m/2})$ in the best case, $O(b^m)$ in the worst case
- Space complexity: $O(bm)$

Alpha-Beta (minimax) Search

- Optimal if the opponent also plays optimally
- Complexity: $O(b^{m/2})$ in the best case, $O(b^m)$ in the worst case
- Space complexity: $O(bm)$

Still unreasonable time complexity for large games (e.g., chess, go)

Factors may improve performance

- *killer moves*: if immediatley optimal (e.g., pruning a large subtree)
- *move ordering*: if the best moves are explored first, more pruning will occur
- *symmetry*: if the game has symmetries, we can avoid exploring symmetric states

Factors may improve performance

- *killer moves*: if immediatley optimal (e.g., pruning a large subtree)
- *move ordering*: if the best moves are explored first, more pruning will occur
- *symmetry*: if the game has symmetries, we can avoid exploring symmetric states

Still unreasonable time complexity for large games (e.g., chess, go)

Shannon: 'Programming a Computer for Playing Chess' (1950)

- Type A: limit depth:
  - *cutoff test*: stop the search at a certain depth, and use an *evaluation function* to estimate the utility of the leaf nodes
  - *quiescence search*: if the leaf node is not 'quiet' (e.g., it is a capture move), continue searching until a 'quiet' node is reached
  - *iterative deepening*: start with a small depth limit, and increase it until time runs out
- Type B: limit breadth
  - *forward pruning*: kill branches that are unlikely to lead to a good moves
  - *beam search*: only keep the best k nodes at each level
  - *Monte Carlo Tree Search*: use random sampling to estimate the value of the nodes, and only expand the most promising ones

Monte Carlo Tree Search

- Used in Go, Chess, and other games
- Combines tree search with random sampling
- At each step, it builds a search tree by:
  - Selection: select a leaf node to expand based on a selection policy
  - Expansion: if the selected node is not a terminal state, expand it by adding one or more child nodes
  - Simulation: simulate a random playout from the new node to a terminal state
  - Backpropagation: update the values of the nodes in the path from the new node to the root based on the result of the simulation

Less simple games

- *Stochastic games*: games with probabilistic transitions (e.g., backgammon)
  - use trees with intermediate 'probability nodes' to model the stochasticity
  - use *expectimax* instead of minimax to compute the values of the nodes
- *Partially observable games*: games where the players do not have full information about the state of the game (e.g., poker)
  - use trees with intermediate 'chance nodes' to model the uncertainty
  - use belief states to represent the player's knowledge about the state of the game

Summary

Book references:

- Chapter 6

Next TA session:

- Exercises on Search [Chapter 3 and 4 so far]

Next lecture:

- Decompose the description of states